

JOB PORTAL WEB APPLICATION

A Full-Stack Implementation using React.js and Java Servlets

PROJECT REPORT



CHANDIGARH UNIVERSITY

Discover. Learn. Empower.

Department of Engineering (CSE)

Submitted by: Lavish

UID: 24BCS12448

Date of Submission: April 24, 2026

TABLE OF CONTENTS

Section	Title	Page No.
1.	PROJECT OVERVIEW	1
2.	FEATURES AND FUNCTIONALITY	2
3.	FLOW OF THE PROJECT	3
4.	CONCLUSION	4
5.	REFERENCES	5

1. Project Overview

1.1 Vision and Core Purpose

The **Job Portal Application** is a professional, full-stack web solution designed to simplify the connection between recruiters and job seekers. In a digital landscape often crowded with complex interfaces, this project focuses on clarity and ease of use. The goal was to build a platform that reduces the "mental effort" required for a seeker to find work while providing recruiters with a powerful, straightforward tool to manage their listings.

Our design follows a "zero-friction" philosophy. By strictly separating the public browsing areas from the private administrative sections, we ensure that the seeker's experience remains fast and uninterrupted by the technical data management happening behind the scenes.

1.2 Structural Organization

To keep the user experience organized and secure, the application is divided into two distinct areas:

- **The Home Page (Public Discovery):** This is the public face of the app. Built specifically for job seekers, it emphasizes readability and a clear visual layout. It includes a "Featured Jobs" section to highlight high-priority roles and an "Available Jobs" list for general browsing. To protect the information, this section is strictly read-only.
- **The Job Management Page (The Administrative Hub):** This is the workspace for authorized users. It provides a full suite of management tools—creating, editing, and deleting jobs—through an intuitive interface that does not require the user to have any technical database expertise.

1.3 Technical Foundation

The project uses a three-tier structure to ensure it is reliable and ready to grow:

- **Frontend (React.js):** React was chosen to give the app a modern, "Single Page" feel. This means the page updates instantly without needing to refresh, making everything from theme switching to job browsing feel seamless.
- **Backend (Java Servlets):** These act as the "brain" of the operation. Servlets process requests from the website and communicate with the database. We use JSON (a lightweight data format) to keep information moving quickly between the front and back ends.
- **Database (MySQL):** All job data is stored in a MySQL database. The system is designed to handle thousands of listings—including titles, company names, locations, and salaries—with high efficiency.
- **Styling (CSS3):** We used modern CSS variables to build a design that is both professional and intelligent. The site is fully responsive, meaning it looks great on any device, and it supports a custom Dark/Light mode engine.

2. Features and Functionality

2.1 A "Seeker-First" Home Page

The **Home Page** is engineered to help users find relevant opportunities in seconds.

- **Hero Section:** A bold, professional header that immediately explains the site's purpose and guides the user on what to do next.
- **Quick Links:** These shortcuts allow users to jump to specific sections instantly, preventing the need for excessive scrolling.
- **Featured vs. Available Jobs:** To help certain roles stand out, we placed a "Featured" section at the top. While this currently displays the newest posts, the system is prepared to support a "featured flag" in the database for manually selected listings.

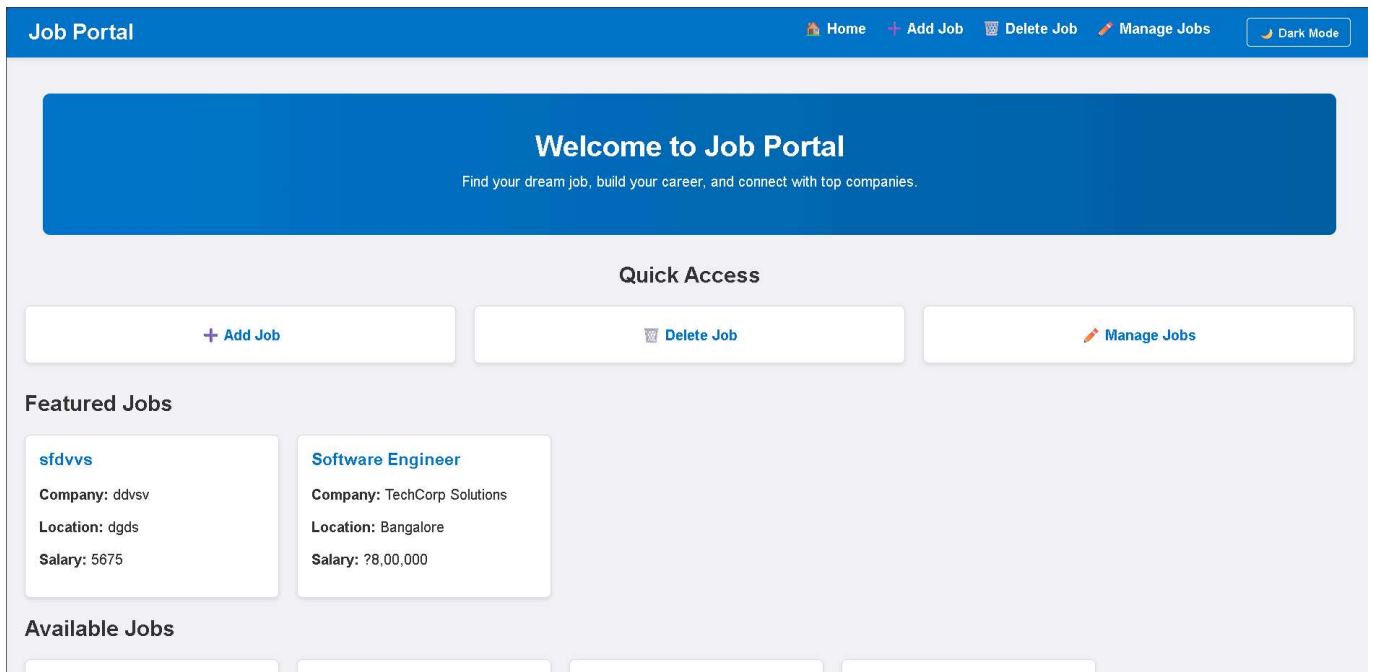


Figure 1: Home Page Hero Section and Navigation

2.2 The Administrative Command Center

The **Job Management Page** is where the portal is kept up to date.

- **Create & Edit:** Admins can quickly open a form to add new jobs or update existing details. These changes are saved instantly to the database.
- **Simple Deletion:** When a role is filled, it can be removed with one click to keep the board clean.
- **Live Updates:** Thanks to the way the code is structured, the list refreshes automatically as soon as an admin makes a change. There is no need to manually reload the page to see the results.

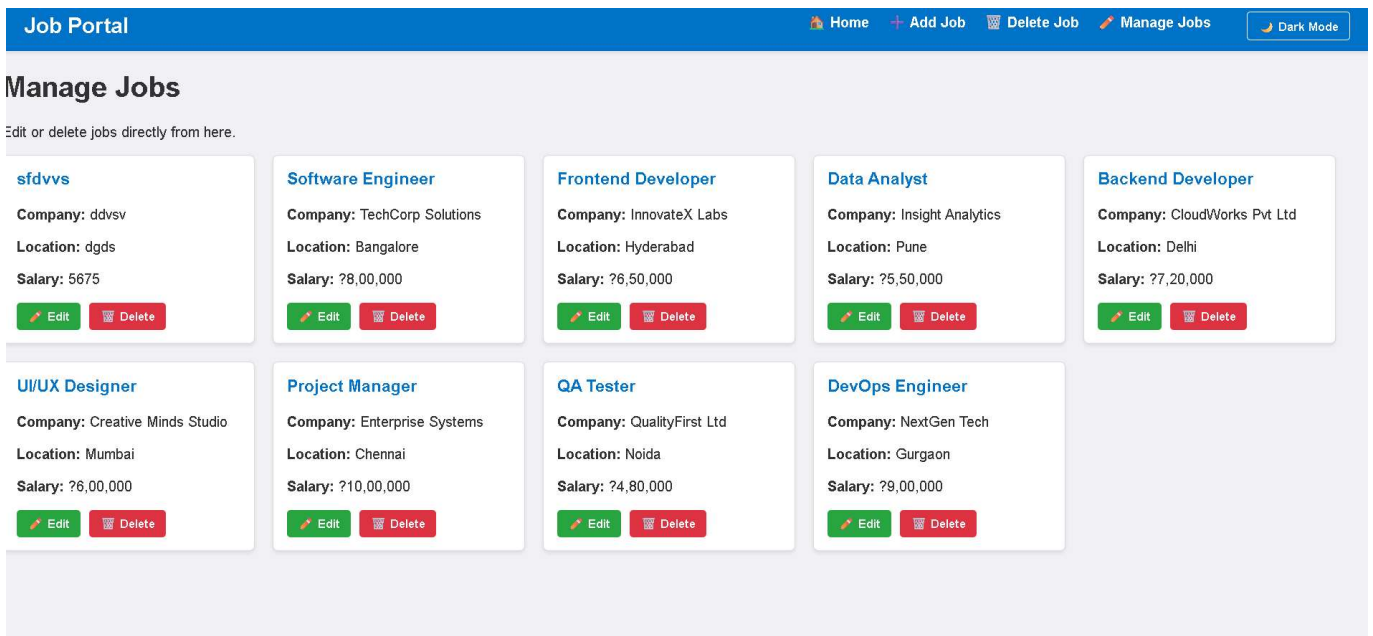


Figure 2: Job Management Page with Administrative CRUD Actions

2.3 Dynamic Theming (Dark/Light Mode)

To ensure the app is comfortable to use at any time of day, we implemented a sophisticated theme toggle.

- **The Logic:** By adding a `.dark-mode` class to the `<body>` of the website, the entire interface adapts. Instead of hardcoding colors, we used "CSS Variables," which act as a master switch for the site's appearance.
- **The Look:** For the Dark Mode palette, we chose a professional purple and teal accent. This provides high contrast but low glare, making it easier on the eyes during late-night job hunting.
- **Accessibility and Future Growth:** This feature is a significant benefit for users with light sensitivity or those working in low-light environments. While the theme currently resets if the page is closed, we plan to add `localStorage` in future versions so the app remembers the user's preference.

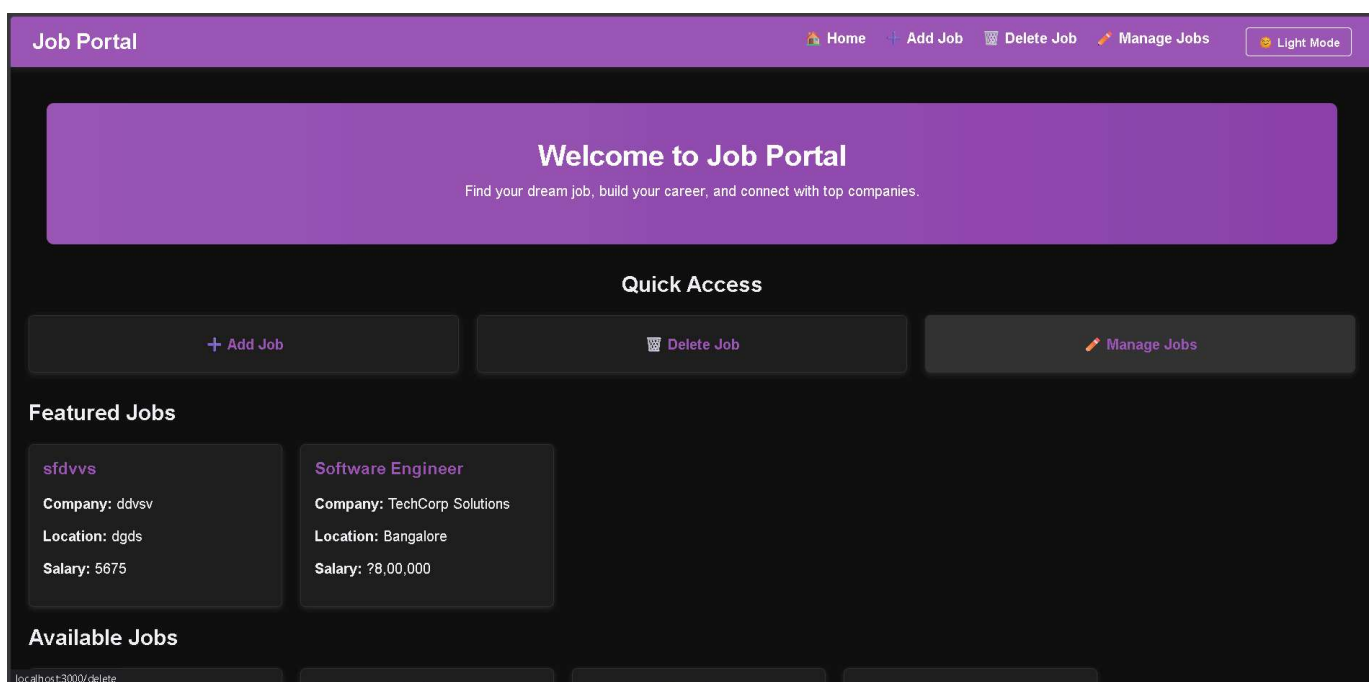


Figure 3: Dark Mode Toggle with Purple/Teal Accents

2.4 Responsive Design

The interface was built with a "Mobile-First" approach. Whether someone is browsing on a small smartphone or an admin is working on a large desktop monitor, the layout shifts smoothly to fit the screen.

3. Flow of the Project

3.1 The User Experience

The journey is designed to be intuitive. A user starts on the **Home Page**, browses through jobs, and uses **Quick Links** to navigate. If the user is an administrator, they move to the **Job Management Page** to perform tasks. Every action taken by the admin sends a request to our Java backend, which then updates the MySQL filing system.

3.2 Data and API Interaction

We built a structured set of "conversations" (APIs) between the user and the database:

- **GET:** Retrieves the job list to show on the screen.
- **POST:** Submits a brand-new job listing.
- **PUT:** Updates the details of a specific job.
- **DELETE:** Permanently removes a listing.

3.3 Component Hierarchy

The frontend is broken into logical building blocks to keep the code clean:

- **App.js:** The main controller that manages the overall theme and page navigation.
- **Home.js & JobManagement.js:** These files handle the specific logic and data for each of the two main pages.
- **JobCard.js:** This is a single, reusable component that adapts to its environment. On the **Home Page**, it stays simple and read-only. On the **Job Management Page**, it automatically displays the "Edit" and "Delete" buttons.

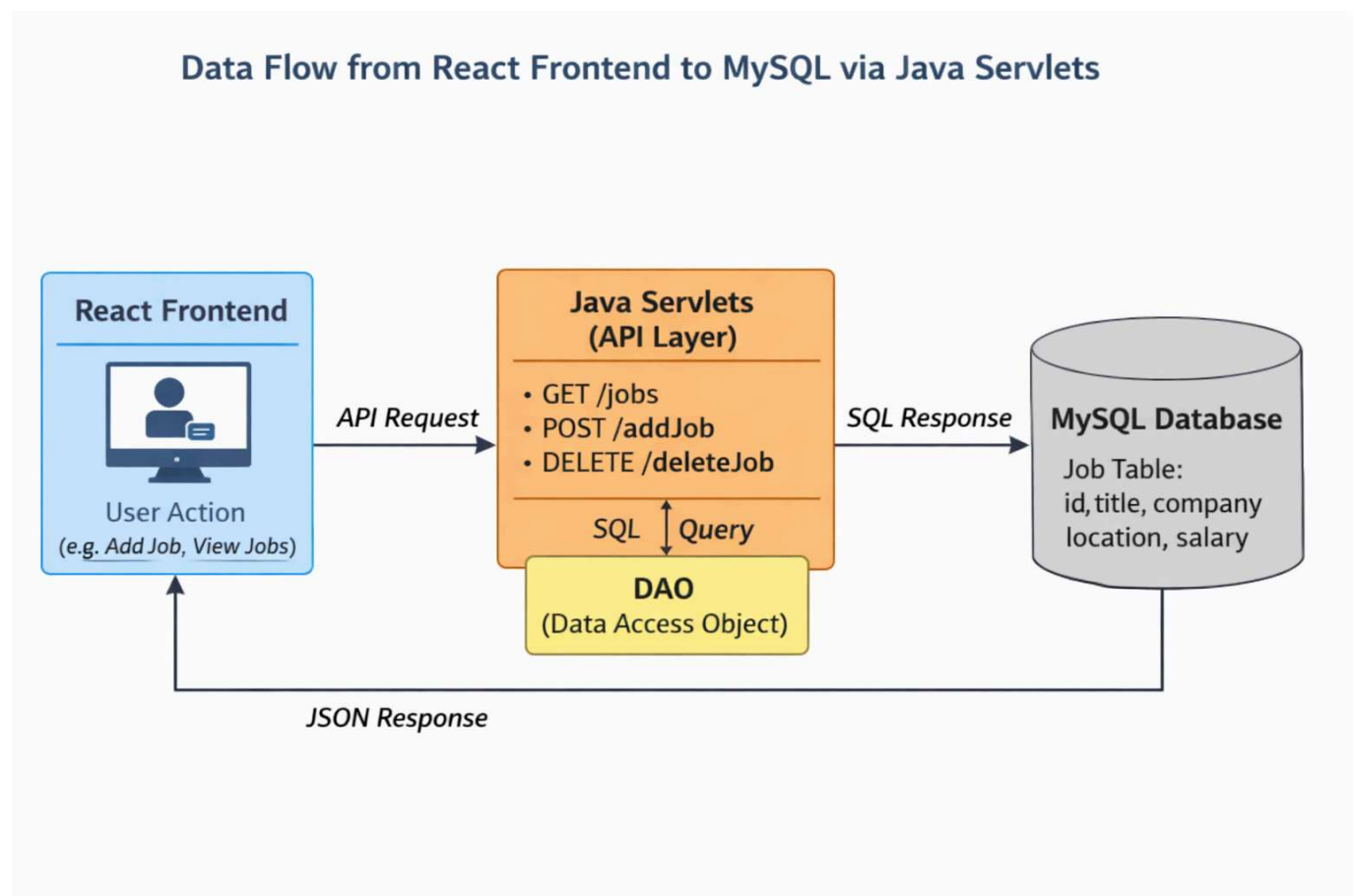


Figure 4: Data Flow from React Frontend to MySQL via Java Servlets

4. Conclusion

4.1 Summary of Achievement

The Job Portal Web Application successfully combines a fast, modern React frontend with a stable Java backend. We have created a platform that looks professional, feels great to use (especially with Dark Mode), and keeps the "browsing" and "managing" sides of the business perfectly balanced.

4.2 Scalability and Growth

Because we used an optimized database and a modular design, this is more than just a basic project—it is built to scale. The system can handle thousands of job postings and many simultaneous users without a dip in performance.

4.3 The Roadmap Ahead

We are excited to continue improving the platform. Future updates will focus on:

- **Secure Logins:** Adding industry-standard security (JWT) to ensure only authorized admins can access the management features.
- **Advanced Search:** Allowing users to filter jobs by salary, location, or seniority levels.
- **Detailed View Pages:** Giving each job its own dedicated page with a full description and an application button.

5. REFERENCES

1. **React.js Documentation** Meta Open Source. *Getting Started with React*. Available at: <https://react.dev/>
(Used for implementing component-based UI, state management, and hooks).
2. **Java Servlet Specification** Oracle Corporation. *Java Servlet Technology*. Available at: <https://docs.oracle.com/javaee/7/tutorial/servlets.htm>
(Used for building the API layer and handling HTTP requests).
3. **MySQL Reference Manual** Oracle Corporation. *MySQL 8.0 Reference Manual*. Available at: <https://dev.mysql.com/doc/refman/8.0/en/>
(Used for database schema design and SQL query optimization).
4. **MDN Web Docs: CSS Custom Properties** Mozilla Social. *Using CSS custom properties (variables)*. Available at: https://developer.mozilla.org/en-US/docs/Web/CSS/Using_CSS_custom_properties
(Used for implementing the Dark/Light mode theme engine).
5. **W3Schools Web Development Tutorials** *Responsive Web Design - Introduction*. Available at: https://www.w3schools.com/css/css_rwd_intro.asp
(Used for ensuring cross-device compatibility and Flexbox/Grid layouts).
6. **JDBC API Documentation** Oracle Corporation. *Java Database Connectivity (JDBC)*. Available at: <https://docs.oracle.com/javase/8/docs/technotes/guides/jdbc/>
(Used for connecting the Java backend to the MySQL database).